A Measured, Reproducible Recipe for Fine-Tuning $\pi_{0.5}$ on All 100 Tasks of the 2026 BEHAVIOR Challenge

Vivek Chandra
Team Dexterity
[TO CONFIRM: affiliation and contact e-mail]

November 2026

Abstract

We fine-tune the $\pi_{0.5}$ vision-language-action model [1] on the 20,000 demonstrations of the 2026 BEHAVIOR Challenge [5]. One model covers all 100 tasks, and its inputs are three RGB cameras and proprioception. The method is a standard one: no architectural changes, no reinforcement learning and no extra data. Every setting was decided by a measurement, or by a rule fixed before the measurement was made. Three findings mattered most. First, re-encoding the training videos at 224×224 *without B-frames* made random-access video decoding about 195× cheaper per file open and raised data-loader throughput from 25 to 82 samples/s. Second, two-node data parallelism only pays off at a large per-GPU batch: 0.82× at 16 samples/GPU, 1.44× at 64. Third, *action compression* at inference (executing 26 predicted steps in 20) doubled the mean task score in a pre-registered paired test on held-out instances, from 0.226 to 0.448. With the official evaluator on the public test instances (853 rollouts), the final model scores **0.339**, with 154 full successes. We release the training code, the serving code, the weights, a Docker policy server, all rollout videos and a step-by-step guide. A fresh account following only the README has run the whole pipeline end to end on a short schedule.

1 Overview

The 2026 BEHAVIOR Challenge asks a single simulated R1Pro mobile manipulator (two 7-DoF arms, a 4-DoF torso and a mobile base) to perform 100 long-horizon household activities in OmniGibson. A rollout's score q is the fraction of the task's goal predicates satisfied when the episode ends, and the overall score is the mean over the 100 tasks. At evaluation time the policy sees only onboard RGB, depth and proprioception, and it must run as a Docker image on one 24 GB GPU.

Our entry is a deliberately conventional system: $\pi_{0.5}$, fine-tuned from the public `pi05_base` checkpoint with the BEHAVIOR fork of openpi. Our contribution is not a new method. It is (i) a training and inference recipe in which every choice is backed by a measurement against an alternative (Table 1), (ii) the systems work that made a 112.5 M-sample run feasible on a shared academic cluster with a 2-day job limit, and (iii) a release built and tested so that someone else can reproduce it (Section 7).

Released materials.

- Code: `https://github.com/VivekChandra1324/behavior-2026-dexterity`. This holds the training *and* serving changes as an overlay on the pinned openpi commit, the data-preparation scripts, the exact run configuration, a training guide, and this report.
- Weights (checkpoint 135,199, with normalisation statistics and serving settings): `https://huggingface.co/VivekChandra/pi05-behavior-2026-dexterity`.
- Policy server: `ghcr.io/vivekchandra1324/b1k-pi05-policy:135199`.
- All 853 evaluator result files and rollout videos: `https://huggingface.co/datasets/VivekChandra/b1k2026-rollout-videos`.

2 Data

The official dataset is in LeRobot v3 format: 100 tasks × 200 teleoperated demonstrations, giving 20,000 episodes and 210,916,774 frames at 30 fps (1,953 hours). Each frame has three cameras (head, left wrist, right wrist) stored as HEVC video (RGB and depth), a 61-dimensional proprioceptive state and a 23-dimensional action: absolute joint targets for base velocity (3), torso (4), each arm (7) and each gripper (1). The median demonstration lasts 5.6 minutes and the longest 25.4. Before training we verified the download four independent ways: byte-exact sizes of all 38,157 files, row counts against `total_frames`, a decode of the last second of each of the 17,093 videos, and checks of the state and action layout.

Depth dropped. Depth is 67 % of the dataset (2.08 TB). Neither official baseline reads it, and $\pi_{0.5}$'s vision tower is pretrained on RGB, so we use RGB and proprioception only.

The B-frame problem. Training samples random frames, so every sample opens a video file and seeks. Our first 224^2 re-encode (libx265, preset `superfast`) inserted B-frames. With B-frames, storage order differs from display order. LeRobot's cheap "approximate" seek assumes the two are the same, and it returned the wrong frame for about 37 % of seeks. The correct "exact" seek scans the file's packet index on every open: 0.821 s against 0.003 s for a file without B-frames, about 195× more per sample. On a 3-task subset the files stayed in the page cache and the cost was invisible. On the full dataset the loader delivered 18.9–25.3 samples/s against the ∼89 the GPUs could consume.

We therefore re-encoded every RGB video once more: area downscaling to exactly 224 × 224 ($\pi_{0.5}$'s input size, which makes the model's own resize a no-op), CRF 28, GOP 8 and `-bframes 0`. We checked the result file by file. All 5,286 files are present, frame counts are identical to the source, the last frame of every file decodes, PSNR is 34–38 dB, and a loader test with the training access pattern returned 0 wrong frames in 300 random seeks. Against the pixels the model saw before, the mean absolute error is 2.73/255. Together with a bounded LRU cache for open decoders (LeRobot's cache never evicted and exhausted host RAM), loader throughput rose from 25 to **82 samples/s**. The training copy is 309 GiB of video plus 70 GiB of low-dimensional data, against 3.3 TB for the original.

No curation. We found 0 exact and 0 near-duplicate trajectories and 200 distinct starting instances per task. Only 0.7 % of frames fall after the annotated task end and 6.4 % are idle. The real imbalance is between tasks, whose shares of all frames range from 0.20 % to 2.47 %. We handle it with task-tempered sampling (Section 3).

3 Model and training recipe

$\pi_{0.5}$ [1] combines a PaliGemma vision-language backbone [3] with a flow-matching [4] action expert, 3.35 B parameters in total. It predicts a chunk of future actions from the three images, the proprioceptive state (discretised into prefix tokens) and a text prompt. We keep the architecture of the BEHAVIOR fork unchanged. The prompt is the task's snake_case name (e.g. `turning_on_radio`), because that is what the fork's `prompt_from_task` reads from the dataset, and the same string must be used at inference.

Every change to the fork is opt-in behind an environment variable, defaults to the upstream behaviour, and has a unit test. Table 1 lists the final settings and the evidence for each. Held-out action error, measured in raw joint units on a fixed exam of 50 tasks × 32 frames (fixed seed) and compared as a *paired* difference, was our comparison metric throughout. Training loss was not used, because the organizers had stated that lower loss does not imply higher task success. Several ideas taken from the two leading 2025 solutions [6, 7] were measured and rejected.

Budget. The step budget was the main lever we had. Our first plan would have shown the model about 40× fewer samples than the 2025 winner's run [6] while covering twice as many tasks. Production therefore ran at a global batch of 832 for 135,200 steps: **112.5 M samples, about 0.53 epoch**.

setting (final)	evidence
initialise from `pi05_base`	a released 2025 checkpoint loaded cleanly but gave the same loss and 2× the gradient norm
15 flow-matching samples per example (one prefix pass, 15 noise/time draws through the action expert)	−8.3 % held-out action error (2 seeds)
isotropic flow noise ($\pi_{0.5}$ default)	correlated noise [6] measured 11.6 % and 5.6 % *worse*
state: quantile normalisation	tie with z-score in action error, but saturated state values fall from 28.6 % to 5.0 %
actions: global z-score	per-time-step normalisation [6] 4.7 % worse on the first 8 chunk steps
vision tower trained	freezing it was 17 % faster per step but 4 % worse
auxiliary temporal-stage head (stage $= \lfloor \text{frame}/(\text{len}/N) \rfloor$, $N = \text{clip}(\text{round}(\overline{\text{len}}/900), 5, 15)$)	no measurable effect either way; the intended 32× larger loss weight was 2.04 % worse (above a pre-set 2 % limit), so kept at the original weight
task sampling $\propto \text{frames}^{0.5}$	the metric weights tasks equally, while uniform frame sampling gives the longest tasks up to 12× the signal of the shortest
peak LR 6.4×10^{-5}, warm-up 1,000, constant, then $(1 - \sqrt{t})$ cool-down over the last 15 %	LR sweep at batch 768: 9.0e-5 → 0.019131, 6.4e-5 → 0.019370, 4.5e-5 → 0.019878; pre-registered rule: one $\sqrt{2}$ step below the best
AdamW ($\beta = 0.9/0.95$), gradient clip 1.0, EMA 0.99	fork defaults

Table 1: Final training settings and the measurement behind each. Errors are paired held-out action errors on the fixed exam.

4 Training at scale on a shared cluster

We trained on two nodes, each with 4 NVIDIA H100 80 GB GPUs, under a 2-day job limit, using JAX with openpi's mesh in an HSDP layout: FSDP over the 4 GPUs inside a node (NVLink) and 2 data-parallel replicas across nodes (InfiniBand).

When two nodes help. The bytes exchanged between nodes per step are constant (∼18 GB), so the speed-up depends only on how much computation hides them:

samples per GPU	16	32	64	FSDP across all 8 GPUs
2-node speed-up	0.82×	1.13×	**1.44**×	0.98×

Production used 104 samples per GPU. Getting the second node to join at all required pinning NCCL and Gloo to the Ethernet interface that actually routes between nodes, and binding the host InfiniBand libraries, which took inter-node bandwidth from 0.12 GB/s over TCP to 12.07 GB/s RDMA.

Surviving the 2-day limit. Each job traps a pre-timeout signal and re-submits itself to resume from the newest complete checkpoint. A successor job is queued ahead of time, and the running job hands over cleanly after a checkpoint save. Incomplete checkpoints are deleted before any resume, and checkpoints are written every 1,000 steps (about the Young–Daly optimum for a ∼40 s save). The resumed data stream is re-seeded per link so that no link replays the previous one's samples.

Production run. The run had seven links, from 28 September to 10 October 2026: about 11 days and ∼2,400 H100-hours. Peak memory was 61.8 of 75.2 GiB per GPU. At clean speed a step took 6.75 s (123.3 samples/s), i.e. 230 TFLOP/s per GPU, or 23.3 % model-FLOPs utilisation. The first link, before the data-path fixes, ran at 8.3 % MFU and lost about 25 % of its time to data stalls. We traced the stalls to three causes in turn and fixed each: (1) the 309 GiB video set did not fit the parallel file system's client cache, and in-order batch delivery let one slow read idle a whole node, so we staged the videos to node-local NVMe and delivered batches out of order; (2) all page cache sat on one NUMA bank, so we used interleaved allocation; (3) loader workers read a 97 GB memory-mapped Arrow table from the network file system, so we staged it in RAM. Held-out error on the fixed exam improved at every link boundary (0.018810 at step 2,000, 0.014845 at 20,000, 0.012572 at 65,000). By a pre-registered rule the peak LR was therefore never lowered early.

5 Inference

The policy server is openpi's WebSocket server with four changes, all in the release:

1. **One server for all 100 tasks**: the prompt follows each request's `task_id`, mapped to the training task names.
2. **Receding-horizon control**: the model's chunk is executed for 20 steps, then the policy replans from the current observation. The evaluator's optional chunk-replay protocol is supported and is unit-tested to give identical actions to per-step requests.
3. **Action compression** [6]: the first 26 predicted steps are resampled with a cubic spline onto 20 control steps, with base velocities scaled accordingly and joint targets not. It is disabled through a grasp, and it makes execution about 1.3× faster.
4. **Gripper re-open rule** [6]: if a gripper is fully closed (on nothing) in a task whose demonstrations never close that gripper, the chunk is replaced by hold-position with the gripper opened. The table is derived mechanically from our demonstrations for all 100 tasks (`scripts/derive_gripper_table.py`, which reproduces the published table byte for byte). It also holds per-stage closure data, but the served policy passes no stage signal, so only this task-level test is used (90 of 100 tasks for the left gripper, 78 for the right).

We checked train/serve consistency item by item:

- the prompt is byte-identical;
- the normalisation statistics come from the checkpoint itself (match to 7.5×10^{-9});
- observation keys and proprioception layout match;
- the server's bilinear resize of the full-resolution render is within 1.7–2.8/255 mean absolute difference of the training frames.

Why compression: a pre-registered paired test. Failure analysis of early rollouts showed the robot doing the right thing too slowly. The head camera was still for ≥1 s during 19–75 % of a rollout, against 1–16 % in the demonstrations, and episodes timed out mid-task. We compared the previous settings (A: replan every 16 steps) with compression plus the gripper rule (B: replan every 20 steps). The test covered 10 tasks × 2 held-out instances (indices 11 and 12; the leaderboard instances 0–9 were never used for testing or tuning), with both arms of each pair on the same machine and the decision rule fixed in advance.

	A: replan every 16	B: compression 26→20 + gripper rule
mean q (20 pairs)	0.226	**0.448**
full successes	2	**8**
pairs better	2	7 (11 ties)

The exact one-sided paired sign-flip test gives $p = 0.051$, and there were no server errors in 40 logs. We adopted B. The gripper rule never fired in the 20 B rollouts, so the gain comes from action compression. This is a small test, and we read it as evidence that execution speed was a binding constraint for our model, not as a precise effect size.

6 Results

We evaluated checkpoint 135,199 once on the public test instances 0–9 with the official evaluator (BEHAVIOR-1K v3.9.3-post1, default robot configuration, the official `RGBDFullResWrapper`, default time limits), using the inference settings above. The run used rented RTX 3090/4090 machines. Our budget did not cover all 1,000 rollouts, and instances not run score 0, so we followed a coverage rule fixed before the run. Instance 0 of every task was run. Any task with a non-zero score on any instance ran all 10. A task stopped after three consecutive zeros. Every completed rollout was submitted, zeros included, and each instance was run at most once (re-run only when the machine failed before producing a result).

Of the 853 rollouts, 154 were full successes, 422 partial and 277 scored zero; 79 tasks scored above zero (Table 2, Figure 1). The best tasks reach 0.85 (`installing_a_modem`, `putting_dirty_dishes_in_sink`). Long

task class (time limit)	tasks	rollouts	full successes	tasks scoring 0	mean task score
short ($\leq$ 8,000 steps)	18	124	39	8	0.297
medium ($\leq$ 15,000)	29	262	58	4	0.372
long ($>$ 15,000)	53	467	57	9	0.335
all	100	853	154	21	**0.339**

Table 2: Self-evaluated results on public test instances 0–9 (task score = mean over 10 instances, missing = 0).

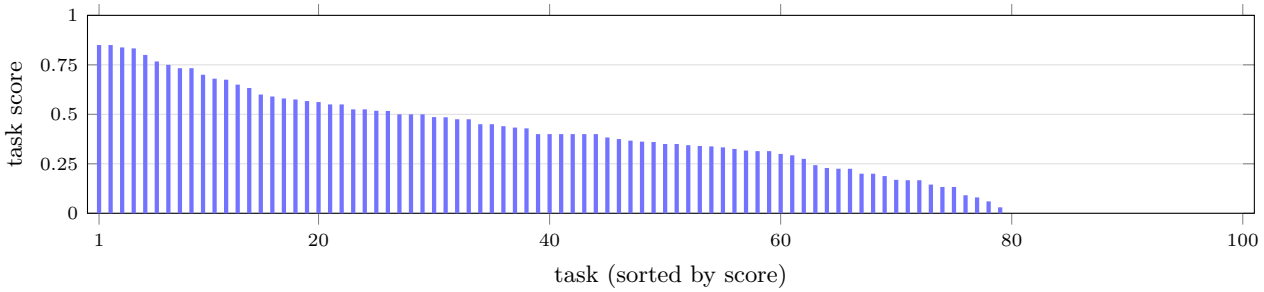


Figure 1: Per-task scores for all 100 tasks. The full table is in `docs/RESULTS.md`.

tasks are not systematically worse than short ones: 8 of the 21 zero-score tasks are short, a class that contains only 18 tasks. Sixteen attempts produced no result and were re-run: 7 simulator physics crashes, 4 server-start faults on a half-copied model, 2 WebSocket drops (fixed by disabling the server's keepalive ping), 2 lost machines and 1 scene-load hang.

As an organizer-style check, we pulled the public image anonymously by digest, ran it as shipped, and connected the newer official evaluator (v3.9.3-post2) from a separate machine. It completed 3 of 3 rollouts successfully on held-out instances, including a task switch within one container and both request modes.

7 Reproducing this work

The repository README is a single Quick start block. It sets up the code (the openpi BEHAVIOR fork at commit `0cc8e355`, plus our overlay of modified files), downloads the dataset, builds and verifies the 224^2 copy, computes the normalisation statistics and stage table, and trains, resuming automatically after any interruption. `configs/train.env` holds the exact settings of the final run with every switch explained. `scripts/serve.sh` serves a checkpoint with the submission's inference settings and is the same command as the Docker image's entrypoint.

Tested as a new user. We fetched the public README anonymously, extracted its Quick start block and ran it line by line on 2 H100 GPUs, inside a container with an empty home directory and a clean environment, logging every unavoidable departure: prerequisites already installed, the dataset download and transcode replaced by links to our verified copies, and a short schedule. The test ran clone, setup, environment install, dataset verification (5,286 files, 0 problems) and asset preparation, then trained from the anonymously downloaded base weights with checkpoints, and resumed correctly when re-run. The release tests found and fixed two bugs: a path mismatch between the guide and the training script, and caches that would have filled a new user's home directory with 105 GB. We also checked that the released serving code *is* the submitted policy. A fresh setup ran `scripts/serve.sh`, the published Docker image ran with its own entrypoint, and both received the same 33 requests (per-step, a task switch, chunk replay). GPU arithmetic is not bit-reproducible across server starts, so we measured that floor by starting the released server twice: up to 3.2×10^{-3} apart (median 2.7×10^{-5}, on actions of order 1). The released code and the image differed by up to 3.6×10^{-3} (median 3.1×10^{-5}), which is within that noise. Both environments have the same 205 packages at the same versions. `scripts/derive_gripper_table.py` rebuilds the published gripper table byte for byte.

Requirements and cost. Training needs at least 2×80 GB GPUs (one works only with the EMA copy disabled), ∼192 GB of host RAM and ∼2 TB of disk. Our full run took ∼2,400 H100-hours. A shorter schedule is a one-line change, and the training guide gives the memory measurements for sizing a run. Serving needs ∼8.5 GB of GPU memory next to the simulator.

8 Lessons and limitations

- **Measure the ceiling before optimising.** An earlier GR00T N1.7 pilot looked loader-bound. Feeding it one cached batch forever, an infinitely fast loader, gained only 5 %, which proved it was compute-bound and led us to $\pi_{0.5}$. $\pi_{0.5}$, in contrast, reached 78 samples/s on cached data but 10.1 samples/s from cold storage: it was input-bound, and the B-frame fix followed.
- **A storage format is a decoding decision.** An encoder preset silently made every training sample 195× more expensive to read. A subset small enough to stay cached hid it.
- **Inference settings can matter as much as training.** The largest measured gain in the project came from executing the same model's actions faster.
- **Limitations.** Our score is self-evaluated, on at most 10 instances per task with 147 instances not run. The paired test has 20 pairs. Most recipe ablations are short runs measured by held-out action error, not by task success. Each ablation was run once or twice. We did not try reinforcement learning, depth input or architectural changes.

Acknowledgements. We thank the BEHAVIOR Challenge organizers for the benchmark, the dataset and the $\pi_{0.5}$ baseline, and the authors of the two leading 2025 solutions for releasing their code and reports. [TO CONFIRM: computing-centre acknowledgement; statement on AI-assistant use]

References

[1] Physical Intelligence. $\pi_{0.5}$: a Vision-Language-Action Model with Open-World Generalization. arXiv:2504.16054, 2025.

[2] K. Black et al. π_0: A Vision-Language-Action Flow Model for General Robot Control. arXiv:2410.24164, 2024.

[3] L. Beyer et al. PaliGemma: A versatile 3B VLM for transfer. arXiv:2407.07726, 2024.

[4] Y. Lipman et al. Flow Matching for Generative Modeling. arXiv:2210.02747, 2022.

[5] C. Li et al. BEHAVIOR-1K: A Human-Centered, Embodied AI Benchmark with 1,000 Everyday Activities and Realistic Simulation. arXiv:2403.09227, 2024 (preliminary version at CoRL 2022).

[6] I. Larchenko, G. Zarin, A. Karnatak. Task adaptation of Vision-Language-Action model: 1st Place Solution for the 2025 BEHAVIOR Challenge. arXiv:2512.06951, 2025.

[7] J. Bai et al. (Team Comet). Openpi Comet: Competition Solution For 2025 BEHAVIOR Challenge. arXiv:2512.10071, 2025.